



I N N O M A T I C S
R E S E A R C H L A B S

Daily Task Questions

Note:

- Once done with any particular task get it reviewed/Evaluated by you mentor
- Don't do tasks in the word document.

Task 1 - Variables:

1. Run all the identifiers and separate valid and Invalid Identifiers?
 - `my_variable = 25`
 - `counter = 40.7`
 - `_total = "Hello"`
 - `123abc = 34`
 - `my variable = 6.7`
 - `@count = "hi"`
 - `for = "True"`
 - `Name123 = 136`
 - `myFunction = True`
 - `_data = False`
 - `if_condition? = "Innomatics"`
 - `class = "Hyderabad"`
2. Why is Python treating them as invalid Identifiers? Explain?
3. Use different identifiers and assign the below values
 - 10
 - -5
 - 3.14159
 - -0.12345
 - 1000
 - "Hello, World!"
 - "Python"
 - "I am learning"
 - "Text type"
 - "String"

- TRUE
- FALSE

Task 2 - Operators:

Arithmetic Operators:

1. Create a Python script that defines two variables, num1 and num2, with any numeric values.
2. Use arithmetic operators (+, -, *, /, //, %, **) to perform the following operations:
 - Add num1 and num2.
 - Subtract num2 from num1.
 - Multiply num1 and num2.
 - Divide num1 by num2.
 - Find the floor division of num1 by num2.
 - Find the remainder when dividing num1 by num2.
 - Calculate num1 raised to the power of num2.
3. You have a floor that is 95 meter long and 58.8 meter wide. Find out the total area using python and print it.

Logical Operator:

1. Create a Python script that defines two boolean variables, is_sunny and is_warm, with appropriate values (True or False).
2. Use logical operators (and, or, not) to create the following conditions:
 - Check if it's both sunny and warm and print the result.
 - Check if it's either sunny or warm and print the result.
 - Check if it's not sunny and print the result.

Comparison Operator:

1. Create a Python script that defines two variables, a and b, with any numeric values.
2. Use comparative operators (==, !=, <, >, <=, >=) to create the following conditions:
 - Check if a is equal to b and print the result.
 - Check if a is not equal to b and print the result.
 - Check if a is less than b and print the result.
 - Check if a is greater than b and print the result.
 - Check if a is less than or equal to b and print the result.
 - Check if a is greater than or equal to b and print the result.

Concatenation and Repetition

1. Take two variables as first_name and last_name ..concatenate them.
2. print "Innomatics" 5 times in the same line.

Task 3

Lists:

1. Create an empty list called numbers. Append the numbers 1, 2, and 3 to the list.
 - Modify the element at index 1 to be 4.
 - Print the updated list.
2. Create a list called fruits with elements 'apple', 'banana', 'orange', 'mango', and 'kiwi'.
 - Create a new list by slicing fruits to include only the citrus fruits ('orange' and 'mango'). *
Create a new list called tropical by slicing fruits to include only the tropical fruits ('banana' and 'kiwi').
 - Concatenate citrus and tropical lists to create a new list called combined. Print combined.
3. Create a list called colors with repeated elements 'red', 'blue', 'red', 'green', 'red'. Print the number of occurrences of 'red' in the list. Remove all occurrences of 'red' from the list. Print the updated list.
4. Create a list called groceries with initial items: 'bread', 'milk', 'eggs', 'butter'. Print the final list of groceries.
 - Add 'cheese' to the end of the list. Remove 'milk' from the list.
 - Insert 'yogurt' at index 1.
5. Write a program to add item 7000 after 6000 in the following Python List:
list1 = [10, 20, [300, 400, [5000, 6000], 500], 30, 40]
6. Now that you have a nested list from the above output. Write a program to extend it by adding a sublist ["h", "i", "j"] in such a way that output will look like the following list below.
Expected Output :
[10, 20, [300, 400, [5000, 6000, 7000,"h", "i", "j"], 500], 30, 40]
7. You have given a Python list below. Write a program to find value 20 in the list, and if it is present, replace it with 200. Only update the first occurrence of the item.
list1 = [5, 10, 15, 20, 25, 50, 20]

Sets:

1. Set Creation Create a Python set containing the first 10 even numbers {2, 4, 6,8,10}. Write a program to display this set.
2. Set Operations Given two sets, write a Python program to find the union, intersection, and difference of these sets.
3. Set Operations and Methods Write a Python program that takes a set of numbers and performs the following operations:

- Add a new number to the set.
 - Remove a number from the set.
 - Check if a specific number is in the set.
 - Find the size of the set.
4. Set Operations with Lists Create a Python list with duplicate elements and convert it into a set to remove duplicates. Then, write a program to count how many duplicates were removed.
 5. Construct a set1 using range () with elements from 500 to 700 elements inside it,
 - Remove element 1000 such that it doesn't throw an error in output.
 - Add elements from 750 to 850 in the set.
 - Say, set2={600,745,824,580}. Perform all set operations.

Task 4

Tuples:

1. Create a Python tuple containing the names of your favorite fruits. Write a Python program to print each fruit name from the tuple.
2. Tuple Operations Given two tuples, write a Python program to concatenate them into a single tuple and then find the length of the resulting tuple.
3. Tuple and Set Conversion Write a Python program that takes a tuple of integers and converts it into a set. Then, find and print the maximum and minimum value from the set.
4. Tuple Immutability Explain the concept of immutability in Python tuples with examples. Discuss why tuples are considered immutable data structures.
5. Tuple Slicing Given a tuple of 10 elements, write a Python program to print the elements from the 3rd to the 7th position using slicing.
6. Tuple and Set Combinations Create a tuple containing the names of your friend's favorite movies and a set containing the names of your favorite movies. Write a Python program to find and display common elements
7. tup=(1,2,3,2,4,5,3,4,2). Count the occurrences of element '2' in the given tuple.
8. Access value 20 from the tuple,
tuple1 = ("Orange", [10, 20, 30], (5, 15, 25))
9. Create a dictionary with key-value pairs for a person's name and age.
 - Add a new key-value pair "city" with the value "Delhi" to the person dictionary.
 - Remove the key-value pair with the key "age" from the person dictionary.
 - Sort the person's dictionary by its keys.

Dictionary:

1. Create a Python program that defines a dictionary to store information about your favorite book. Include the title, author, publication year, and genre as key-value pairs. Print out the dictionary and do at least 5 dictionary methods on it.
2. Write a Python program that simulates a simple dictionary for translating English words to Spanish. Define a dictionary containing a few English words as keys and their corresponding Spanish translations as values. Allow users to enter an English word, and then display its Spanish translation if it exists in the dictionary.(use google for English to Spanish translator)
3. Create a Python program that stores the grades of five students in a dictionary. Allow users to input the names of the students and their respective grades. Calculate and display the average grade of the students.
4. Build a basic contact list using a dictionary. Initially, the dictionary can be empty. Allow users to add new contacts with names and phone numbers. Users should also be able to search for contacts by name and display their phone numbers(use in operator).
5. Write a Python program that defines a dictionary containing the names of fruits as keys and their corresponding colors as values (e.g., 'apple': 'red').Do at least 5 dictionary methods on it.
6. Update the first set with items that don't exist in the second set.
s1={150,200,400}
s2={200,750,650}
Expected output: s1={150,400}

Task 5

STRINGS:

s=" \$python is fun, python is easy, but python is tough when you don't practice it revise it@ "

1. How many characters are present in the above string
2. split the string using commas(,)
3. count the frequency of "python"
4. count the frequency of "is"
5. check the +,-,*,/,// of 3 and 4 results
6. replace "when " with "if and only if"
7. find the difference between the number of characters in the original string and the number of characters when you replace "python " with "data science". replace only the last occurring python with data sciences
8. read a string dynamically (user input) if your string is having odd number of characters print: true otherwise print : False eg: Innomatics : o/p : False, datascience : o/p : True
9. Ask user to enter a website name and check whether it is valid or not(True/False)

10. Create a string containing a sentence that includes both single and double quotes.
 - Use escape characters to include quotes within the string.
 - Print the modified string.
11. Declare a string with a backslash \ character in it.
 - Demonstrate the use of the escape character to print the backslash itself.
 - Print the resulting string.
12. Construct a string containing newline and tab escape sequences.
 - Print the string to display the formatted output.
13. sentence = "I want to become a Data Scientist".
 - Access and print the first character of the string using indexing.
 - Access and print the last character of the string using negative indexing.
 - What are possible ways to extract Scientist(use forward and reverse)
 - Use slicing to extract and print the first word from the string.
 - Use slicing to extract and print the last three characters of the string.
14. name = "Innomatics Reserach Labs"
 - Apply slicing to extract a substring that includes the second to fifth characters.
 - Print the extracted substring.
15. string = "Python@1234"
 - Use slicing to extract every second character from the string.
 - Print the resulting sliced string.
 - print last five letters.
 - Utilize slicing to reverse the word order in the string.
 - Print the string with the words in the reversed order.
16. a = "123123123"
 - Apply slicing to extract only the numeric characters at odd positions.
 - Print the sliced result.
17. name = "Innomatics", course = "Data Science"
 - write python code in different ways to show output as : "Hello World I am learning Data Science in Innomatics"
 - Replace Science with Analysis in the above output.
18. string = ' Hello World I am learning Python in Innomatics '
 - How many characters are there in the string?
 - Slice only 'Python' from above string
 - Remove white spaces on both sides then find no.of characters in the string.
 - Convert the string in Upper case
 - What is the index number of 'World' in the above string.
 - Split the string with respect to whitespace.after that store that split into a variable and find the data type of that variable.
 - How many times 'a' is repeated in the above string.
19. Create a variable to store the string "Earth revolves around the sun"
 - Print "revolves" using slice operator
 - Print "sun" using negative index
20. Create two variables to store how many fruits and vegetables you eat in a day.

- Now Print "I eat x veggies and y fruits daily" where x and y presents vegetables and fruits that you eat everyday.
21. Create 3 variables to store street, city and country, now create address variable to
- store the entire address. Use two ways of creating this variable, one using + operator and the other using f-string.
 - Now Print the address in such a way that the street, city and country prints in a separate line.

Task 6

Conditional Statements:

1. Basic If-Else Statements Write a Python program that takes an integer as input from the user and prints whether it is multiple of 3 or not using if-else statements.
2. Nested If Statements Create a program that asks the user for their age and nationality. Depending on their age and nationality, display a different message using nested if statements. For example, if they are under 18 and from the USA, display "You are a minor from the USA."
3. Grade Calculator Write a program that takes a student's score as input and converts it to a letter grade (A, B, C, D, or F) using if-elif-else statements. Use the following grading scale:
A: 90-100
B: 80-89
C: 70-79
D: 60-69
F: Below 60
4. Leap Year Checker Create a Python program that checks if a given year is a leap year or not. Use conditional statements to determine whether the year is divisible by 4, 100, and 400 to decide if it's a leap year.
5. BMI Calculator Write a program that calculates a person's Body Mass Index (BMI) based on their height and weight. Then, use conditional statements to categorize the BMI as Underweight, Normal, Overweight, or Obese. Provide guidance on the healthy range for BMI.(use BMI Formula)
6. Calculator with Menu Build a simple calculator program that provides a menu with options for addition, subtraction, multiplication, and division. Take user input for two numbers and an operation choice. Use conditional statements to perform the selected operation and display the result.
7. Temperature Converter Write a program that converts temperatures between Celsius and Fahrenheit. Allow the user to choose the conversion direction and input the

temperature value. Use conditional statements to perform the conversion based on the user's choice.(Do with the conversion formula).

Task 7 - Loops:

1. Print Numbers from 1 to N: Write a Python program that uses a for loop to print all numbers from 1 to N and subtract 5 from each number.where N is a positive integer entered by the user.
2. Calculate the Sum of Numbers from 1 to N: Write a Python program that calculates and prints the sum of all numbers from 1 to N, where N is a positive integer entered by the user, using a for loop.
3. Print Even Numbers: Write a Python program that prints all even numbers from 50 to 100 using a for loop.
4. Reverse a String: Write a Python program that takes a string as input and uses a for loop to print the characters of the string in reverse order.
5. Count Vowels in a String: name = "innomatics research labs" Write a Python program that uses a for loop to count the number of vowels (a, e, i, o, u) in the string. Ignore case sensitivity.
6. Generate a Multiplication Table: Write a Python program that generates and prints the multiplication table for 6.
7. Write a Python program that takes an integer 'n' as input and prints the following pattern:
For n = 4, the pattern should look like this:

```
1
1 2
1 2 3
1 2 3 4
```

8. Create a Python program that generates and prints a multiplication table for a given number 'n'. The table should include values from 1 to 10. For example, if n = 5, the output should be:

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

9. Write a Python program that takes an integer 'n' as input and prints the following patterns:
 - Right-angled triangle pattern:

```

*
**
***
****
*****

```

- Inverted right-angled triangle pattern:

```

*****
****
***
**
*

```

- Create a Python program that takes an integer 'n' as input and prints a number pyramid as shown below: For n = 4, the output should be:

```

1
212
32123
4321234

```

10. write a program that prints the numbers from 1 to 10, each on a new line.
11. write a program that prints the even numbers from 1 to 20, each on a new line.
12. Write a program that calculates and prints the square of each number from 1 to 5.
13. write a program that prints the multiples of 3 from 3 to 30, each on a new line.
14. Write a program that prints the numbers from 10 to 1 in descending order, each on a new line.
15. Modify the program to print only the even numbers from 1 to 20 except multiple of 3 using a while loop.
16. Create a program that prompts the user for an integer input and then uses a while loop to print the countdown from that number to 1.
17. Create a program that uses a while loop to print the numbers from 1 to 20, but skip printing the numbers 5, 10, and 15 using the continue statement.
18. Write a Python program that simulates a simple password entry system. Ask the user to enter a password, and use a while loop to repeatedly prompt for the password until the correct password "abc123" is entered. Once the correct password is entered, exit the loop.
19. Calculate and print the sum of all the positive integers entered by the user.

Comprehensions:

1. Basic List Comprehension: Write a Python program that uses list comprehension to generate a list of cubes of the first 10 positive integers.
2. Filtering with List Comprehension: Given a list of integers, create a new list using list comprehension that contains only the multiple of 3 and 5 from the original list.
3. String Manipulation with List Comprehension: Given a list of strings, create a new list using list comprehension that contains the lengths of each string.

4. List Comprehension with Conditionals: Given a list of numbers, create a new list using list comprehension that contains the squares of even numbers.

Task 8 - Functions:

1. Write a function called "add_numbers" that takes two integers as input and returns their sum.
2. Write a function called "multiply_numbers" that takes two integers as input and returns their product.
3. Write a function called "calculate_average" that takes a list of numbers as input and returns the average of those numbers.
4. Write a function called "is_even" that takes an integer as input and returns True if the number is even, and False otherwise.
5. Write a function called "reverse_string" that takes a string as input and returns the reverse of that string.
6. Write a function called "count_vowels" that takes a string as input and returns the number of vowels (a, e, i, o, u) in that string.
7. Write a function called "find_max" that takes a list of numbers as input and returns the maximum value in that list.
8. Write a function called "find_min" that takes a list of numbers as input and returns the minimum value in that list.
9. Write a Python function to find the maximum of three numbers.
10. Write a Python function to sum all the numbers in a list.
11. Write a Python function to check whether a number falls within a given range.
12. Write a Python function that takes a list and returns a new list with distinct elements from the first list.
13. Write a Python function that accepts a string and counts the number of upper and lower case letters

Built-In-Functions:

1. Create a lambda function that takes two arguments and returns their sum.
2. Create another lambda function that takes a single argument and returns its square.
3. Use the lambda functions to calculate the sum and square of various numbers and print the results.
4. Say, a=27, b=15. Create a lambda function that multiplies argument 'a' with argument 'b' and prints the result.
5. Define a list of integers.
6. Use the filter function to create a new list that contains only the even numbers from the original list.
7. Use the filter function again to create a new list that contains only numbers greater than 10 from the original list. Print both filtered lists.

8. Using in-built function filter() with a lambda function filter even numbers from a given list.
num = [17, 20, 32, 45, 53, 61, 72, 84, 97]
9. Define a list of strings where each string represents a number (e.g., ["1", "2", "3"]).
10. Use the map function to convert each string to an integer and store the result in a new list.
11. Use the map function again to square each integer in the new list and store the result in another new list. Print both new lists.
12. Create a list of dictionaries, where each dictionary represents a person with keys "name" and "age". Example: [{"name": "Alice", "age": 30}, {"name": "Bob", "age": 25}, ...]
13. Use the filter function to create a new list that contains only the people who are older than 25.
14. Use the map function to create a new list that contains the names of the filtered people. Print the list of names.
15. Create a program that reads a list of numbers from the user (you can use the input function for this).
16. Use map and filter functions to perform the following operations:
 - Square each number.
 - Filter out numbers that are not divisible by 3.
 - Print the result
17. Use map() with a lambda function to square each element of a given list.
list1 = [32,45,67,83,2,4,8,10]
18. Find the product of all numbers in the list using reduce and lambda functions.
nums = [15, 2, 31, 4, 56]

Task 9 - File Handling:

1. Load the file '**about_python.txt**' into your jupyter notebook.
 - Correct wrong statements about python features
 - How many characters present in the data
 - How many times the word 'Python' is repeated.
 - After correcting all ..print complete valid features of the python.
2. Create a file '**File.txt**' which explains python file handling whatever you have learnt today(creating,reading,writing and appending...etc) in detail.That reload the saved file(i.e File.txt) to jupyter notebook.
3. Create a file Write a Python program that creates a text file named "**data.txt**" and add data about python programming (should contain at least 5 lines)
4. Read and Display File Contents Write a Python program that reads a text file named "**data.txt**" and displays its contents on the screen.
5. Count Words in a File Write a Python program that reads a text file named "data.txt" and counts the total number of words in the file. Display the word count on the screen.
6. Search and Replace Create a Python program that reads a text file named "data.txt" and asks the user to enter a word to search for. If the word is found in the file, replace all occurrences of the word with a new word provided by the user.

7. Append to a File Write a Python program that prompts the user to enter a sentence. Append this sentence to an existing file named "**notes.txt**". Make sure the newly added sentence starts on a new line.

Exceptional Handling:

1. In this assignment, you will create a Python program to calculate the average grade of students in a function. However, the program should handle various exceptions that might occur during user input and grade calculation. Requirements: provided). c. NegativeValueError: If any student's marks are negative.
2. Your program should prompt the user to enter the total number of students.
 - For each student, prompt the user to enter their name and three subject marks (out of 100).
 - Calculate the average marks for each student and display their names along with their average marks.
 - Calculate and display the overall class average.
 - Implement a loop so that the user can re-enter data if an exception occurs or continue calculating for another class.
 - Include a graceful exit message when the user decides to stop.
3. Define two custom exception classes:
 - NegativeValueError: This exception should be raised when any of the subject marks are negative.
 - InvalidNumber Error: This exception should be raised when the user enters an invalid number for the total number of students (e.g., a negative number).

Task 10 - OOPS:

1. Write a Python program to create a 'Vehicle' class with static variables max_speed and mileage instance attributes. Print the attributes.
Example output:
max_speed = 240
mileage = 18
2. Create a Python class called 'Car' with attributes 'make' and 'model'. Instantiate an object of this class and print its attributes.
3. Create a class for arithmetic operations using different methods like addition(), subtraction() etc.
4. Define a base class 'Vehicle' with attributes color and model. Create a derived class 'Car' that inherits from 'Vehicle' and adds an attribute num_doors.

Example:

```
class Vehicle:
```

```
    def __init__():
```

```

        self.color

        self.model

class Car(Vehicle):

    def __init__():

        super().__init__()

        Self.num_doors

```

Example output- Car: MG Hector, Red; Doors: 4

5. Create two base classes. One base class is 'Person' with attributes name and age. Another class is 'Address' with attributes city and country. Define a derived class 'Employee' that inherits from both Person and Address. In 'Employee' class add an attribute job.

Example:

```

class Person:

    #code

class Address:

    #code

class Employee(Person, Address):

    #code

```

Example Output:

Employee: Sirisha, Age: 25, City: Delhi, Country: India, Job: Data Scientist

6. Create a base class **Animal** with a method **make_sound()**. Define two subclasses, **Dog** and **Cat**, that inherit from **Animal** and override the **make_sound()** method to print the respective sounds.
7. Create a function that can calculate the sum of two numbers or concatenate two strings. The function should be able to take any number of arguments and also demonstrate polymorphism based on the different types of arguments.

Code:

```

class Calculate:
    def addi(self, *args):
        if not args:
            return None
        result = args[0]
        for arg in args[1:]:
            result += arg
        return result

calc = Calculate()
print(calc.addi())
print(calc.addi(3, 5))
print(calc.addi(4.5, 5.5, 1.0))
print(calc.addi("Hello, ", "World", "!"))
print(calc.addi([1, 2], [3, 4], [5, 6], [9, 10]))

```

8. Explain what operator overloading is and provide an example using a custom class.
9. Create a **Person** class with private attributes `__name` and `__age`. Implement getter and setter methods for accessing and modifying these attributes.
 - Modify the **Person** class from the previous problem to use properties for accessing and modifying the private attributes.
10. Create a **BankAccount** class with a protected attribute `_balance`. Implement a method `get_balance()` to retrieve the balance.
 - Create a Python module **calculator** with a private function `__add` that adds two numbers. Implement a public function `add_numbers` that calls the private function.
11. Create an abstract class **Shape** with an abstract method `calculate_area()`. Define two subclasses, **Rectangle** and **Circle**, that inherit from **Shape** and implement the `calculate_area()` method.